第6章 第2节 Function Calling的本质与角色

- 来源: https://class.imooc.com/lesson/5275#mid=104926
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第6周 Function Calling：工具调用机制
- 模块: Function Calling的本质
- 课节: 图文 1-2 Function Calling的本质与角色

1-2 Function Calling的本质与角色

第6章 第2节 Function Calling的本质与角色

> [!TIP] 阅读指南 上一节了解了 Function Calling 能让 AI 从"只会说"变成"能做事"。这一节深入它的技术原理——LLM 如何判断该调用哪个函数、怎么提取参数，先从核心概念讲起：LLM 需要解决什么问题、谁在调用函数、三方角色如何分工。

2.1 LLM需要解决的三个问题

让 LLM 调用函数时，它需要解决三个关键问题：

问题1：我需要调用工具吗？
- 判断用户的请求是否需要外部工具
- 示例：
    "今天天气真好" → 不需要工具（闲聊）
    "今天天气怎么样？" → 需要工具（查询）

问题2：调用哪个工具？
- 从可用工具列表中选择合适的
- 理解每个工具的功能描述
- 示例：
    可用工具：[get_weather, send_email, set_reminder]
    用户："明天会下雨吗？" → 选择：get_weather

问题3：传什么参数？
- 从用户输入中提取参数
- 转换为正确的数据类型
- 处理缺失的参数
- 示例：
    用户："明天上海天气"
    提取：city="上海", date="明天"

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

预览

2.2 不同视角理解Function Calling

从AI的角度看

Function Calling是一个分类+信息抽取任务：

分类+信息抽取

```
# 步骤1：分类任务
用户输入 → 判断意图 → [需要工具] or [不需要工具]

# 或者：多分类任务（如果需要工具）
用户输入 → 判断意图 -> 工具列表 → 选择工具 → [工具A] or [工具B] or [工具C]

# 步骤3：信息抽取任务
用户输入 + 工具定义 → 提取参数 → {param1: value1, param2: value2}
```

从开发者的角度看

Function Calling是LLM阅读工具的说明书，然后LLM告诉你该调用哪个：

```
# ═══════════════════════════════════════
#  你给LLM一份"工具说明书"
# ═══════════════════════════════════════

tools = [
    {
        "name": "get_weather",
        "description": "获取指定城市的天气",  # ← LLM靠这个判断
        "parameters": {
            "city": {"type": "string", "description": "城市名"},
            "date": {"type": "string", "description": "日期"}
        }
    },
    {
        "name": "send_email",
        "description": "发送邮件",
        "parameters": {...}
    }
]

# ═══════════════════════════════════════
#  LLM看完说明书，告诉你调用哪个
# ═══════════════════════════════════════

用户问："明天上海天气怎么样？"

# LLM的思考过程：
# 1. 用户在问天气
# 2. get_weather的描述是"获取天气" ✓ 匹配
# 3. send_email是"发送邮件" ✗ 不匹配
# 4. 决定：调用get_weather
# 5. 提取参数：city="上海", date="明天"

# LLM输出：
{
    "tool": "get_weather",
    "arguments": {"city": "上海", "date": "明天"}
}

# ═══════════════════════════════════════
#  你拿到LLM的决定后，真正执行
# ═══════════════════════════════════════
```

```
result = get_weather(city="上海", date="明天")
```

从系统的角度看

Function Calling是一个多轮对话流程：

多轮对话

2.3 到底是谁在调用函数

一个常见的误解

很多人第一次接触Function Calling时，会有这样的误解：

```
错误：AI自己调用函数
用户 → LLM → LLM自己调用天气API ← 不是这样！


1
2
```

这是不对的！ LLM并不会真正执行函数调用。

这是不对的！

谁才是执行者

```
✓ 正确理解：AI告诉你调用哪个函数，你来执行
用户 → LLM → LLM说"你去调get_weather"
      → 你的代码执行get_weather
      → 你把结果给LLM
      → LLM生成回答
```

归结起来就是三句话：LLM 做决策（判断调用哪个工具、提取参数），代码做执行（真正调用函数），LLM 再做整合（基于结果生成回答）。

2.4 角色分工

LLM的工作

```
# LLM只做两件事：

1. 判断：分析用户意图，决定该调用哪个函数
   用户："明天上海天气"
   LLM：需要调用get_weather

2. 抽取：从用户输入中提取参数
   LLM：city="上海", date="明天"
```

代码的工作

```
# 你的代码负责：

1. 定义有哪些工具可用
   tools = [{"name": "get_weather", ...}]

2. 接收LLM的决定
   tool_name = response.tool_calls[0].function.name
   arguments = response.tool_calls[0].function.arguments

3. 真正执行函数调用（重点！）
   result = get_weather(**arguments)  # ← 这是你调用的！

4. 把结果返回给LLM
   final_answer = llm.chat(tool_result=result)
```

为什么这样设计

安全性

如果LLM能直接调用函数，风险太大：

```
# 危险场景
def delete_all_data():
    """删除所有数据"""
    db.execute("DROP DATABASE production")

# 如果LLM误判 → 直接删库 → 灾难！
```

交给代码来调用，就有了控制权——加确认机制、做权限检查、记录每次操作日志、遇到危险命令直接拒绝。每一步都有兜底。

灵活性

实际执行还可以根据情况灵活调整：

```
# LLM说调用get_weather
# 但你可以根据情况灵活处理：

if user.is_vip:
    result = call_premium_weather_api()  # VIP用高级API
else:
    result = call_free_weather_api()     # 普通用户用免费API

# 或者加缓存
if cache.has(city, date):
    result = cache.get(city, date)  # 命中缓存
else:
    result = call_weather_api()     # 真正调用
    cache.set(city, date, result)   # 保存缓存
```

技术限制

LLM运行在云端（OpenAI/阿里云的服务器），它的物理位置决定了边界——访问不到本地文件，调用不了内网API，更操作不了数据库和硬件设备。这些它都够不着。

这就是为什么叫"Function Calling"而不是"Function Execution"：

- Calling = 呼叫/请求（LLM的角色）
- Execution = 执行（你的代码的角色）

理解了这一点，后面的内容就清晰多了。

2.5 ■ 学点英语

中文	English	音标	说明
函数描述	Function Description	/ˈfʌŋkʃn dɪˈskrɪpʃn/	工具定义中的description字段，LLM靠它判断是否调用该工具
参数提取	Parameter Extraction	/pəˈræmɪtər ɪkˈstrækʃn/	LLM从用户输入中识别并提取函数参数的子任务
决策与执行分离	Decision-Execution Separation	/dɪˈsɪʒn ɪgˈzekjuːʃn ˌsepəˈreɪʃn/	LLM做决策、代码做执行的核心架构原则

图片

